

高级程序设计（计算机科学与技术系、2023-6-18）

一、简述题（本题满分25分）

1. 什么是类？什么是对象？二者之间的关系是什么？（5分）
2. 为什么说继承与封装存在矛盾？C++是如何处置这个矛盾的？（4分）
3. 多继承的使用场景和优势是什么？多继承会额外引用哪些问题？C++是如何解决这些问题的？（5分）
4. STL中迭代器的作用是什么？（4分）
5. 自定义类型转换操作符可能会造成什么问题？如何解决？（3分）
6. C++中的友元是什么？有哪些特性？（4分）

二、程序分析题（本题满分40分）

1. 下面的程序有几个错误，请分别指出。

```
#include <iostream>

class A
{
    int a1;

protected:
    int a2;

public:
    static int si;
    const int ci;

    A() : ci(6)
    {
        a1 = 1;
        a2 = 3;
    }

    static void func_x(int x)
    {
        x = x + a2;
        std::cout << x << std::endl;
    }
};
```

```

int A::si = 6;

class B : protected A
{

public:
    void func_a(int x) const
    {
        x = x + a1;
        a2 = x;
    }

    int get_a2()
    {
        ci = a2;
        return a2;
    }

    void func_a2(const B c2)
    {
        a2 = c2.get_a2();
    }
};

int main()
{
    A c1;
    std::cout << c1.si << std::endl;
}

```

2. 写出下面程序的运行结果。

```

#include <iostream>
using namespace std;

void f(int x);
void g(int x);
void h(int x);

int main()
{
    f(3);
    f(1);
    f(2);
}

```

```
    try
    {
        f(2);
        f(1);
        f(4);
    }

    catch (int)
    {
        cout << "main-int" << endl;
    }

    catch (char)
    {
        cout << "main-char" << endl;
    }
}
```

```
void f(int x)
{
    try
    {
        g(x);
        cout << "f-ok" << endl;
    }
    catch (int)
    {
        cout << "f-int" << endl;
    }
}
```

```
void g(int x)
{
    try
    {
        h(x);
    }
    catch (int)
    {
        cout << "g-int" << endl;
    }
    if (x == 1)
    {
        throw 2;
    }
}
```

```

    }
}

void h(int x)
{
    if (x == 4)
    {
        throw 'a';
    }
    if (x == 3)
    {
        throw -1;
    }
    return;
}

```

3. 写出下面程序的运行结果。

```

#include <iostream>
using namespace std;

class A
{
public:
    A() { cout << "A默认构造\n"; }
    A(const A &a) { cout << "A拷贝构造\n"; }
    ~A() { cout << "A析构\n"; }
};

class B : public A
{
public:
    B() { cout << "B默认构造\n"; }
    B(const B &b) { cout << "B拷贝构造\n"; }
    ~B() { cout << "B析构\n"; }
};

class C
{
public:
    C() { cout << "C默认构造\n"; }
    C(const C &c) { cout << "C拷贝构造\n"; }
    ~C() { cout << "C析构\n"; }
}

```

```

};

class D : public C
{
    B b;

public:
    D() { cout << "D默认构造\n"; }
    D(const D &d) : C(d) { cout << "D拷贝构造\n"; }
    ~D() { cout << "D析构\n"; }
};

int main()
{
    cout << "===stmt1===\n";
    D d1;
    cout << "===stmt2===\n";
    D d2(d1);
    cout << "===stmt3===\n";
    return 0;
}

```

4.下面程序能否正确编译运行？如果能，请写出运行结果；如果不能，请指出错误。

(1)

```

#include <iostream>
using namespace std;

class A
{
public:
    void f() { cout << "A::f()\n"; }
    void f(int i) { cout << "A::f(int)\n"; }
    void g() { cout << "A::g()\n"; }
};

class B : public A
{
public:
    void f(int i) { cout << "B::f(int)\n"; }
    void h()
    {
        f();
    }
}

```

```

        f(1);
        g();
    }
};

int main()
{
    B b;
    b.h();
    return 0;
}

```

(2)

```

#include<iostream>
using namespace std;

class Figure
{
public:
    virtual void draw() const = 0;
    virtual void move() = 0;
};

class Ellipse : public Figure
{
public:
    void draw() const { cout << "Ellipse::draw()\n"; }
};

class Circle : public Ellipse
{
public:
    void move() { cout << "Circle::move()\n"; }
};

int main()
{
    Figure *e1 = new Ellipse;
    Figure *c1 = new Circle;
    e1->draw();
    c1->draw();
}

```

(3)

```
#include<iostream>
using namespace std;

class A
{
public:
    A() { f(); }
    virtual ~A() { f(); }
    virtual void f() { cout << "A::f()\n"; }
    void g() { cout << "A::g()\n"; }
    void h()
    {
        g();
        f();
    }
};

class B : public A
{
public:
    B() { f(); }
    ~B() { f(); }
    void f() { cout << "B::f()\n"; }
    void g() { cout << "B::g()\n"; }
};

int main()
{
    cout << "===stmt1===\n";
    A *p = new B;
    cout << "===stmt2===\n";
    p->f();
    cout << "===stmt3===\n";
    p->A::f();
    cout << "===stmt4===\n";
    p->g();
    cout << "===stmt5===\n";
    p->h();
    cout << "===stmt6===\n";
    delete p;
}
```

三、实现下面的两道程序（本题满分35分）

1.函数式编程中，规约（Reduce）可根据给定函数将一个列表合并成一个值。如下的模板函数reduce有三个形式参数，分别为规约函数、规约初始值以及待规约的列表，reduce返回规约后的结果。reduce有两个模板参数，分别为规约函数的类型F以及待归约值的类型T。请补全代码，使程序正确运行，允许使用任意C++语言标准和标准库。

```
template <typename T, typename F>

T reduce(F func, T init, const std::vector<T> &args)
{
    // Your Code
}

int main()
{
    std::vector<int> numbers = {1, 2, 3, 4, 5};
    int sum = reduce([](int a, int b) { return a + b; }, 0, numbers);
    std::cout << "Sum: " << sum << std::endl;
    int product = reduce([](int a, int b) { return a * b; }, 1,
numbers);
    std::cout << "Product: " << product << std::endl;
    return 0;
}
//程序输出：
//Sum: 15
//Product: 120
```

2.在课堂上我们学习了std::string，它内部维护一个char数组以及相关元数据（长度、容量）来表示字符串。在真实的STL中，std::string往往被实现为写时复制，即让多个字符串共享相同的数据，只在需要修改时才进行复制。以下代码给出了一个具体实现CowString，每一个CowString会维护一个引用计数refcount，并按如下规则更新refcount：

- a) 通过const char* 构造时，refcount值初始化为1；通过拷贝构造时，refcount初始化为传入的CowString的refcount并加1，将str指向传入的CowString的str；
- b) 调用赋值函数时，将当前refcount值减1，之后规则同拷贝构造；
- c) 当析构时，将refcount值减1；
- d) 任何时候，如果refcount为0，析构str以及ref_count;

e) 当CowString调用set方法时，如果refcount大于1，意味着此时有其他CowString也使用了底层的str，不能直接在当前str上直接修改，需要重新初始化当前CowString的refcount以及str_。

请你补充以下代码，使其满足以下要求，并使测试代码的意图被正确执行。允许使用任意C++语言标准和非容器标准库，也可以使用字符串拷贝函数strcpy、字符串长度函数strlen以及字符串比较函数strcmp。(30分)

- (1) 正确实现const char* 构造函数、拷贝构造函数
- (2) 正确实现析构函数，不能存在内存泄露、重复析构等问题
- (3) 正确实现operator==
- (4) 正确实现operator=
- (5) 正确实现CowStringProxy中的operator=函数
- (6) 正确实现operator[]函数，若访问的下标超出了字符串长度，则抛出异常(含有错误信息"Out Of Range")
- (7) 支持将CowString中的维护的字符串输出到cout流，无需换行

若不确定某些功能是否应该实现，只需支持测试代码中的用法，无需考虑过于复杂的情况；但不允许针对测试数据做特判。

```
class Exception
{
public:
    virtual std::string what() = 0;
};

class CowString
{
public:
    CowString(const char *src);
    CowString(const CowString &other);
    ~CowString();
    class CowStringProxy
    {
    public:
        CowStringProxy(size_t idx, CowString &cow_str) : idx_(idx),
cow_str_(cow_str) {}
        CowStringProxy &operator=(const char &ch); // Your Code
        operator char() const { return cow_str_.str_[idx_]; }
```

```

private:
    size_t idx_;
    CowString &cow_str_;
};
char *c_str() const { return str_; }

private:
    size_t *ref_cnt_;
    size_t size_;
    char *str_;
    void init_ref() { ref_cnt_ = new size_t(1); }
    void dec_ref()
    {
        (*ref_cnt_)--;
        if (*ref_cnt_ == 0)
        {
            delete ref_cnt_;
            delete[] str_;
        }
    }
    void inc_ref() { (*ref_cnt_)++; }
    size_t get_ref() const { return *ref_cnt_; }
    // TODO
};

int main()
{
    CowString foo("test");
    CowString bar = foo;
    CowString test("test");
    std::cout << (foo == bar);
    std::cout << (foo.c_str() == bar.c_str());
    std::cout << (foo == test);
    std::cout << (foo.c_str() == test.c_str());
    foo[1] = 'f';
    foo[1] = 'e';
    std::cout << (foo == bar);
    std::cout << (foo.c_str() == bar.c_str());
    std::cout << (foo == test);
    std::cout << (foo.c_str() == test.c_str());
    std::cout << foo;
    return 0;
}
//程序输出: 11101010test

```

